

RapidBoxForest: Link-Envelope-Guided Box Forests for Low-Overhead Scene Construction in Multi-Query Manipulation

Yu Tian and Hongliang Ren

Abstract—Repeated-query manipulation benefits from reusable planning structure, but constructing and re-certifying general collision-free convex regions can be costly in cluttered workcells. RapidBoxForest (RBF) uses axis-aligned joint boxes as the reusable region primitive: it checks boxes with endpoint-to-link swept-capsule bounds, organizes boxes that pass the configured scene-validation policy into a scene-level forest and typed corridor graph, and stores kinematics-keyed envelope evidence in a Lifelong Envelope Cache Tree (LECT) separately from scene labels. Certificates are conditional: they apply only to conservative box unions or finite-cover OBB-corridor unions (OBB: oriented bounding box); paths that are repaired, shortened, post-processed, or derived from filter-only evidence require independent query validation. Saved-catalog experiments use audit-separated timing and explicit cache accounting. On Shelf+IIWA, warm-cache depth-23 LECT replay gives a 0.166 s median reusable build, 0.130 s median Online/q (per-query online time), and 0.164 s median five-query amortized time; live HIFK-5 (hierarchical interval forward kinematics) materialization takes 7.006 s. On saved random scenes, RRT-Connect has lower median Online/q in five of six cases but worse audited path ratios in all six; RBF has lower median Online/q than BIT* (Batch Informed Trees) in all six, while BIT* has better ratios in the four UR5/Panda cases. In a maintenance-only scene-cache study, leaf-sweep insertions show a 153.3–207.5 $\times$ speedup and removals show a 943.6–3354.0 $\times$ speedup relative to a 15-obstacle warm scene-cache rebuild. Snapshot creation and loading costs, along with resident memory, are outside the reported timing rows; update rows report maintenance only, not the full update-and-query pipeline. The open-source implementation is available at <https://github.com/tianyu1997/RapidBoxForest>.

Index Terms—Motion planning, multi-query planning, link interval envelopes, box-region planning, configuration-space planning.

I. Introduction

Many high-degree-of-freedom manipulation workloads contain related queries in nearly fixed workcells. Robot kinematics persist, while start-goal pairs, local obstacles, or small scene edits change. Convex-region planners, including IRIS (Iterative Regional Inflation) and Graphs of Convex Sets (GCS), provide optimization-ready region-level abstractions once collision-free regions are certi-

fied [1]–[3], but region construction can be costly in high-dimensional or frequently edited scenes. Sampling-based planners, including PRM and RRT variants, avoid such region construction [4]–[7], but their reusable structures are typically zero-volume roadmaps or query trees.

RBF occupies a scoped middle ground between these families. It grows axis-aligned joint-space boxes, checks them with link-envelope bounds, connects boxes that pass the configured scene-validation policy into a typed corridor graph, and reuses that graph across later queries. The trade-off is explicit: boxes are less expressive than more general convex covers, but they simplify validation, graph building, updates, and multi-query reuse.

a) Problem statement: Let $\mathcal{Q} \subset \mathbb{R}^n$ be the joint-limit box of a fixed manipulator and let $\mathcal{O} \subset \mathbb{R}^3$ be the current workspace obstacle set. Given a set or stream of queries $(q_s, q_g) \in \mathcal{Q}^2$, RBF builds a finite family of configuration-space (C-space) boxes that pass the configured scene-validation policy, called policy-accepted boxes below, and an adjacency graph over those boxes. The design goal is to reduce scene-construction and maintenance overhead while retaining reusable volumetric planning structure and conservative evidence where the stated validation policy supports it.

The geometric core adapts classical endpoint bounding. For rounded links, endpoint envelopes bound endpoint trajectories over a joint box; their radius-expanded convex hull encloses the swept link and can certify C-space boxes when the endpoint source and separation tests are conservative. Conservative sources yield conditional box certificates, whereas tighter non-certifying sources remain filters whose paths require query validation. Reuse is split accordingly: scene-level RBF caches store one scene’s box forest and corridor graph, while the Lifelong Envelope Cache Tree (LECT) stores replayable kinematics-keyed endpoint and link evidence.

The evaluation mirrors this certificate boundary. Success is counted only after fixed-step query validation unless a row is explicitly conservative. Warm-cache Shelf rows include replay-based scene construction and per-query retrieval; they exclude snapshot creation and loading costs, resident memory, and offline setup amortization. Dynamic update rows measure scene-cache maintenance, not the full update-and-query pipeline. Thus the experiments evaluate reusable-region construction under stated validation and cache-accounting boundaries, not claims of domi-

The authors are with the Department of Electronic Engineering, The Chinese University of Hong Kong (CUHK), Hong Kong SAR, China. E-mail: ty1997@link.cuhk.edu.hk.

TABLE I
Qualitative taxonomy of reusable manipulation-planning paradigms. The comparison emphasizes the persistent object, validation and certificate granularity, and scene-edit semantics rather than ranking planners by runtime.

Family	Persistent object	Vol.	Validation and certificate unit	Scene-edit reuse	Main trade-off
PRM and LazyPRM	Roadmap vertices and edges	No	Local edges or path segments	Topology can be reused; affected edges require rechecking	Reusable multi-query graph, but reusable evidence is zero-volume.
RRT-Connect and RRT	Query tree	No	Incremental path segments	Little persistent reuse across queries	Query-focused discovery; persistent structure is query-local.
BIT* and FMT*	Batch sample graph or tree	No	Candidate graph edges	Primarily query-centric; limited scene-level reuse	Anytime quality improvement, but still edge-centric.
Adaptive cell decomposition	Configuration-space cells	Yes	Cells under exact, sampled, or interval tests	Explicit cells can be locally relabeled when maintained	Volumetric representation; scalability depends on the cell validator.
IRIS and GCS	Certified convex regions and graph	Yes	Convex collision-free regions	Region reuse is scene-coupled; edits may require reinflation or recertification	Optimization-ready regions with construction and recertification cost.
RBF	Box forest + typed graph + LECT evidence	Yes	Joint boxes via link envelopes; query validation for filter-only paths	Scene boxes are maintained locally; LECT replays kinematics-keyed evidence	Volumetric reuse via axis-aligned boxes; certificates are conditional.

Abbrev.: PRM=probabilistic roadmap; RRT=rapidly exploring random tree; BIT*=Batch Informed Trees; FMT*=Fast Marching Tree; IRIS=Iterative Regional Inflation; GCS=Graphs of Convex Sets; LECT=Lifelong Envelope Cache Tree; Vol.=volumetric.

nance over single-query planners, repair-free certification, or completeness. This work contributes:

- 1) *Box validation*: endpoint-to-link envelopes provide conditional box certificates when their inputs are conservative; otherwise they are used only as query-validated filters.
- 2) *Planning object*: policy-accepted boxes form a reusable scene forest and typed corridor graph.
- 3) *Two-level reuse*: scene caches hold boxes and graph state; LECT replays kinematics-keyed envelope evidence.
- 4) *Local scene-cache maintenance*: deletions promote blocked domains; insertions trigger bounded refill.
- 5) *Evaluation protocol*: saved catalogs, audit-separated timing, and cache accounting define what is and is not compared.

Sections III–VI cover link envelopes, LECT, forest construction and updates, and evaluation.

II. Related Work

Convex-region planners construct collision-free convex regions and search or optimize on the induced graph. IRIS variants inflate collision-free convex regions, and Graphs of Convex Sets (GCS) make the resulting graph optimization explicit [1]–[3], [8]–[10]. RBF shares the region-level objective but uses axis-aligned C-space boxes; this restricted geometry limits expressivity while keeping validation, adjacency, and update operations local and explicit. Table I summarizes the qualitative position.

a) Continuous-collision-detection endpoint bounds: Swept-volume and interval-kinematics methods supply the geometric primitives used here. Convex-hull generator bounds and capsule radius lifts provide swept-set enclosures, while the Proximity Query Package (PQP), Gilbert–Johnson–Keerthi (GJK), the Flexible Collision Library (FCL), and interval Denavit–Hartenberg propagation provide collision-query and interval-kinematics primitives used in related continuous-collision-detection

pipelines [11]–[22]. Redon *et al.* combine such primitives for articulated-body continuous collision detection over time steps [23], [24]; related Taylor-model and safety methods use similar swept-sphere bounds [25]–[27]. RBF uses joint-box intervals rather than trajectory time steps as the bounding domain, stores compatible interval evidence in LECT, and assembles policy-accepted boxes into a typed corridor graph. Repaired, shortened, post-processed, or filter-only paths remain subject to query validation.

b) Reuse and adaptive decomposition: Multi-query planners amortize effort through reusable graphs. Probabilistic roadmaps (PRMs) and LazyPRM reuse roadmaps and edge checks [6], [28]–[30], experience-based systems reuse prior paths [31], and adaptive cell methods split cells until the accepted-cell graph can be searched [32], [33]. RBF combines reusable graph search with box-level envelope validation and a separate kinematics-keyed evidence cache.

c) Sampling baselines and refinement: RRT*, RRT-Connect, Batch Informed Trees (BIT*), Fast Marching Tree (FMT*), and the Open Motion Planning Library (OMPL) provide standard single-query or anytime implementations and benchmarking infrastructure [7], [34]–[39]. Trajectory optimizers refine supplied paths [40]–[43]. RBF is complementary: it supplies reusable volumetric context before repair, smoothing, and query validation rather than replacing those stages.

III. Link Interval Envelopes

The link-envelope validation layer maps joint regions to endpoint and link bounds; graph connectivity and query acceptance are handled by later layers. For a joint region $I \subseteq \mathcal{Q}$, endpoint envelopes $E_k(I)$ bound endpoint trajectories and link envelopes $B_\ell(I)$ bound rounded links over the same region:

$$I \mapsto \{E_k(I)\} \mapsto \{B_\ell(I)\}.$$

Axis-aligned joint boxes are the default regions; local oriented bounding boxes (OBBs) in scaled joint coordi-

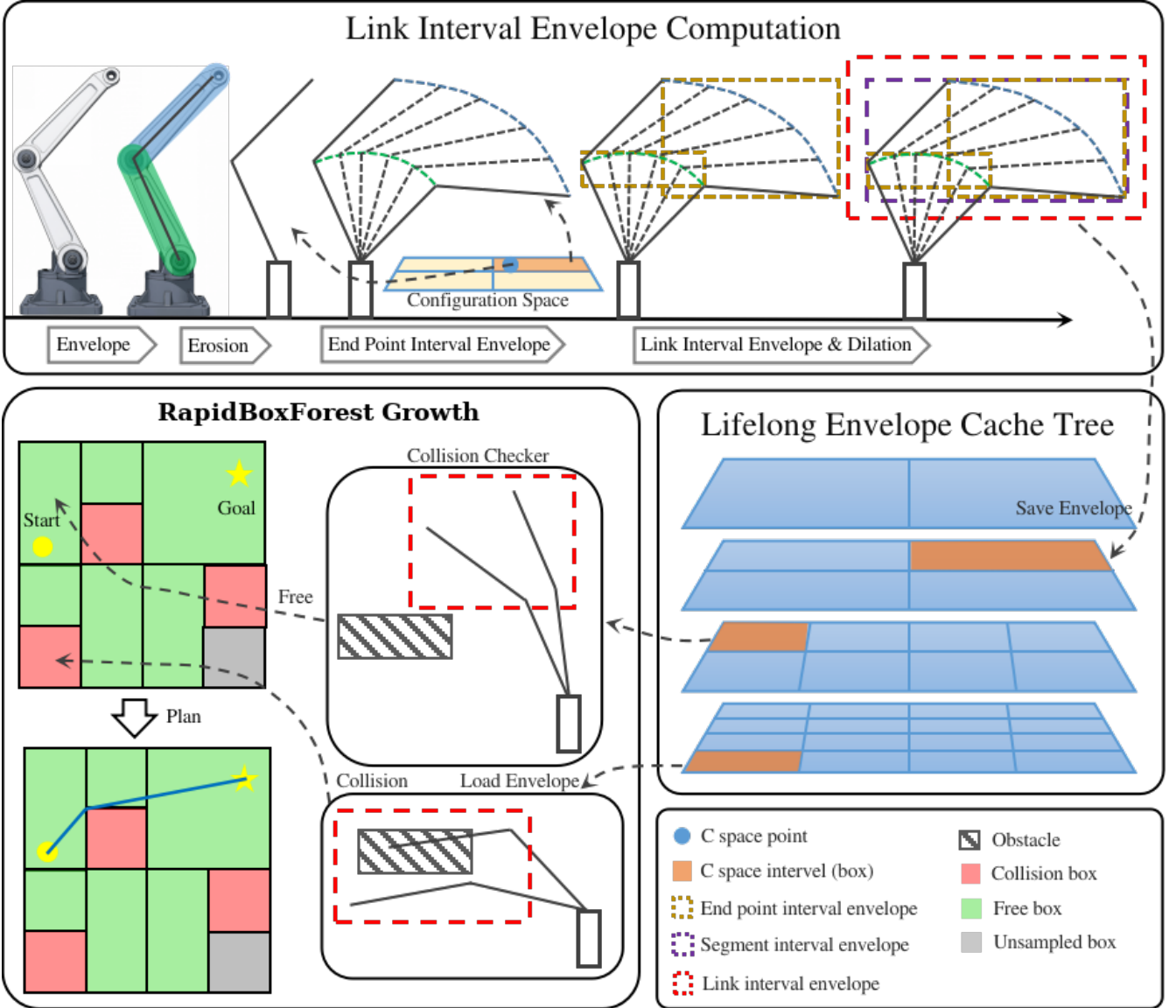


Fig. 1. RBF pipeline for link-envelope-guided multi-query planning. Top: endpoint interval envelopes induce link interval envelopes over a C-space box. Middle: LECT reuses kinematics-keyed endpoint and link-envelope records across repeated builds. Bottom: scene filtering and sample-guided growth expand a policy-accepted scene-level box forest for repeated corridor queries.

nates are used only for typed bridge corridors. Conservative endpoint evidence supports certificates only after link construction and scene-separation tests. Empirical or otherwise non-conservative endpoint sources are filters; candidate edges or paths that depend on them require query validation.

Assumption 1 (Collision model for certificates). *The certificate model is limited to collision between active rounded links and $\mathcal{O}$; joint limits are enforced by requiring $I \subseteq \mathcal{Q}$. Self-collision, attachments, grippers, task regions, and other constraints are outside this certificate model unless encoded in the validation policy and cache descriptor. The reported experiments instantiate this subset: active-link versus workspace-obstacle checks.*

A. Geometric Foundation

Let $T(q)x = R(q)x + t(q)$ be the transform at configuration q , and let $\mathcal{S}_I(C_0)$ denote the workspace set swept by body C_0 as q ranges over I . For a capsule $C = S \oplus B_2(r)$, the fixed radius ball commutes with rigid motion:

$$\mathcal{S}_I(C) = \mathcal{S}_I(S) \oplus B_2(r).$$

Thus a rounded link can be bounded by first bounding the zero-radius center segment and then lifting by the link radius. For a convex body $P = \text{conv}(v_1, \dots, v_m)$, outer bounds on the generator trajectories also bound the swept body,

$$\mathcal{S}_I(P) \subseteq \text{conv}\left(\bigcup_i \hat{\Gamma}_i(I)\right).$$

For a capsule link, only the two endpoint trajectories are needed. Let $\Gamma_a(I)$ and $\Gamma_b(I)$ denote the exact workspace trajectories of the proximal and distal endpoints over

I. RBF uses these swept-sphere bounds over a joint-box domain and stores the resulting bounds as reusable kinematics-keyed bound records.

Proposition 1 (Endpoint envelopes induce link interval envelopes). *Consider a rounded link $L = [a, b] \oplus B_2(r)$ and a joint interval I . If endpoint envelopes $E_a(I)$ and $E_b(I)$ satisfy $\Gamma_a(I) \subseteq E_a(I)$ and $\Gamma_b(I) \subseteq E_b(I)$, then*

$$\mathcal{S}_I(L) \subseteq \text{conv}(E_a(I) \cup E_b(I)) \oplus B_2(r).$$

Consequently, any conservative outer representation of the right-hand side is a valid link interval envelope for the whole joint box I .

Proof. For each $q \in I$, the center segment of the moved link is the convex hull of the two moved endpoints, and those endpoints lie in $\Gamma_a(I)$ and $\Gamma_b(I)$. Hence all center segments are contained in $\text{conv}(\Gamma_a(I) \cup \Gamma_b(I))$. The endpoint inclusions place this set inside $\text{conv}(E_a(I) \cup E_b(I))$. Adding the fixed radius ball commutes with the rigid-motion sweep of the capsule, giving the stated containment. $\square$

Corollary 1 (Conservative joint-box validation). *Let $I \subseteq \mathcal{Q}$ be a joint box and let $\{B_\ell(I)\}$ be conservative link interval envelopes for all active rounded links. If $B_\ell(I) \cap \mathcal{O} = \emptyset$ for every link ℓ , then every configuration $q \in I$ is collision-free with respect to the collision model in Assumption 1.*

Proof. By Proposition 1, each conservative link envelope contains the swept workspace volume of its link over all configurations in I . Disjointness from $\mathcal{O}$ for every active link therefore excludes workspace obstacle collision for the whole manipulator at every $q \in I$ under Assumption 1. $\square$

a) *Numerical conservatism:* A certificate is attached only when endpoint propagation, radius padding, and obstacle-disjointness tests are outward bounds under Assumption 1. Axis-aligned bounding box (AABB) padding uses a conservative B_∞ radius lift. SupportHull/GJK is certificate-backed only with outward obstacle inflation, explicit margins, and conservative termination tolerances. Empirical endpoint sources and post-processing outside a conservative box union or a finite-cover OBB-corridor union are treated as non-conservative filters; they do not extend Corollary 1 or Theorem 1.

B. Endpoint Interval Envelopes

For endpoint k , let $\Gamma_k(I)$ be its exact trajectory over interval I . Any conservative set containing $\Gamma_k(I)$ is an endpoint interval envelope; the basic record is an endpoint AABB because it is compact and directly reusable by all link-envelope representations. The implementation uses two certificate-backed endpoint sources. IFK-AA denotes interval forward kinematics (IFK) with affine-arithmetic bounds propagated along the Denavit–Hartenberg chain; shared noise symbols preserve part of the trigonometric correlation. Hierarchical IFK (HIFK) bisects the joint interval and forms hulls of child endpoint boxes, reducing wrapping at higher materialization cost. IFK is the unsplit

affine-arithmetic pass; HIFK is a hierarchical split-and-hull evaluation (for example, HIFK-5 means five split levels), so the split depth is part of the cache descriptor and cannot be exchanged with an unsplit record. A critical-sample source probes interval endpoints, midpoints, near-boundary configurations, and trigonometric turning candidates; it can reduce over-approximation relative to IFK, but does not certify coupled extrema. An analytical extrema source solves low-dimensional sinusoidal boundary subproblems, reducing one- and two-joint boundary cases to structured root search. It becomes conservative only with an interval-closure wrapper; the Analytical row in Table V is otherwise a diagnostic reference. Without such a wrapper, critical-sample and analytical sources are diagnostics in the endpoint study and filters when used by the planner, not certificate evidence. Any path using either source is accepted only after fixed-step query validation unless wrapped by an independent conservative certificate. During LECT tree descent, transient prefix transforms are reused so unaffected kinematic-chain prefixes are not recomputed; this is a runtime materialization optimization rather than persistent scene state. Fig. 1 shows the endpoint-to-link pipeline, LECT reuse, and box-forest use.

C. C-space OBB Endpoint Envelopes

Axis-aligned boxes remain the global forest primitive. Bridge repair motions can introduce joint correlations. For a bridge transition to be certificate-supported, RBF validates a local typed C-space OBB-corridor region with the finite-cover test below, while leaving the dyadic forest unchanged. Without a successful finite cover, the same transition can be used only as a typed candidate that requires query validation. OBBs are defined in scaled joint coordinates. Let $y = S(q - q_{\text{ref}})$, where S is a diagonal joint scaling, and let

$$R_{\text{obb}} = \{q_c + A\xi \mid \xi \in \Xi\},$$

$$\Xi = \prod_{i=1}^m [-1, 1], \quad A = S^{-1}U \text{diag}(h),$$

with U orthonormal in the scaled metric and h the halfwidth vector. The object R_{obb} is a configuration-space region, not a workspace OBB: the certificate comes from endpoint and link envelopes over R_{obb} , not from the fitted orientation.

a) *Cover certificate:* The conservative cover test tiles the local cube; for each tile $X_r \subseteq \Xi$, it validates the joint-space AABB hull

$$I_r = \text{bbox}_q(q_c + AX_r).$$

If every I_r passes the standard conservative link-envelope scene test, then

$$R_{\text{obb}} \subseteq \bigcup_r I_r \subseteq \mathcal{C}_{\text{free}},$$

where $\mathcal{C}_{\text{free}}$ is the collision-free configuration set under Assumption 1. When this test succeeds, the stored OBB certificate is a finite cover of ordinary box certificates.

b) *Correlated endpoint envelope*: A tighter source keeps the local OBB variables ξ correlated through the forward-kinematics computation:

$$q_j = q_{c,j} + a_j^\top \xi, \quad \xi_i \in [-1, 1],$$

where $a_j^\top$ is the j -th row of A . Affine-arithmetic propagation then produces endpoint records

$$E_k(R_{\text{obb}}) = c_k + \sum_i g_{k,i} \epsilon_i \oplus [-\rho_k, \rho_k].$$

Equivalently,

$$E_k(R_{\text{obb}}) = p_k(q_c) + J_k(q_c) A \Xi \oplus [-\rho_k, \rho_k],$$

where each coordinate remainder is bounded conservatively, for example by

$$\rho_{k,\mu} \geq \frac{1}{2} \sum_{a,b} |(A^\top H_{k,\mu} A)_{ab}|,$$

with $H_{k,\mu}$ an interval or otherwise conservative Hessian bound over R_{obb} . If these remainders are sampled rather than outward bounded, the OBB source is a non-conservative filter and not a certificate.

OBB endpoint records feed the same endpoint-to-link construction as joint boxes. These records refine a local typed edge; they do not add OBB nodes to the global axis-aligned forest. For a zonotope-plus-box endpoint record,

$$h_{E_k}(d) = d^\top c_k + \sum_i |d^\top g_{k,i}| + \sum_{\mu=1}^3 |d_\mu| \rho_{k,\mu}.$$

The zero-radius link centerline support is

$$h_\ell^\circ(d) = \max\{h_{E_a}(d), h_{E_b}(d)\},$$

and the rounded-link envelope adds the capsule radius,

$$h_\ell(d) = h_\ell^\circ(d) + r_\ell \|d\|_2.$$

Coordinate-axis queries of h_ℓ provide an AABB broadphase, while the same support record can be used by the SupportHull/GJK narrow phase. An OBB-corridor edge is certificate-supported only when its endpoint bounds, finite-cover test, and separation tests are conservative; otherwise it remains a typed transition requiring query validation. OBB-corridor records belong to the scene-local typed graph, not the persistent dyadic LECT tree.

D. Link-Envelope Representations

Endpoint records can be exported to multiple link representations without changing the upstream kinematic source. For a capsule link, $B_\ell(I) = B_\ell^\circ(I) \oplus \mathcal{R}(r_\ell)$, with $B_\ell^\circ(I)$ bounding the centerline sweep and $\mathcal{R}$ the radius lift.

a) *AABB envelope*: AABB envelopes wrap the two endpoint envelopes in one box and pad by the link radius:

$$B_\ell(I) = \text{bbox}(E_{\ell-1}(I) \cup E_\ell(I)) \oplus B_\infty(r_\ell).$$

Segment subdivision can retain a union of padded sub-envelopes for more selective filtering, but the union must remain explicit to preserve the tighter bound.

b) *Fixed-direction slab envelope*: A discrete oriented polytope record with k directions (k -DOP) stores lower and upper support values along a fixed direction set. Endpoint boxes are projected onto those directions, padded by the link radius, and merged over retained segment pieces. In the runtime pipeline, these slab tests form a low-cost intermediate filter: after the padded AABB broadphase, non-axis directions can reject close obstacle candidates before GJK. Slab tests are filters unless their endpoint source and separation tests are conservative.

c) *SupportHull envelope*: SupportHull keeps the two endpoint AABBs and link radius as a compact support record. For direction d , the support point is selected from the proximal or distal endpoint box, and obstacle refinement uses a fixed-size GJK narrow phase with outward obstacle inflation. The support record preserves more directional structure than a single AABB while keeping the cached record compact. When present, fixed-direction slab records act only as an intermediate filter between the AABB broadphase and SupportHull/GJK refinement; they do not change the certificate condition, which still depends on conservative endpoint evidence and outward scene tests.

IV. Lifelong Envelope Cache Tree

LECT replays geometry below the scene-level RBF cache; it does not cache scene-validity decisions. It stores scene-independent endpoint and link-envelope evidence keyed by robot kinematics and joint intervals. Obstacle tests, box labels, graph connectivity, and query outcomes remain scene-local, and every replayed envelope is checked against the current obstacle set.

A. Cache Key and Split Policy

LECT is a dynamically grown binary k -d tree over $\mathcal{Q}$. A node stores its joint interval, split metadata, cached endpoint evidence, and runtime residency state. The central invariant is that evidence is *kinematics-keyed* rather than *scene-keyed*: replay imports geometry bounds, not scene-validation labels or graph edges.

a) *Evidence/label separation*: An envelope record is replay-compatible only when the robot model, endpoint source, envelope representation, split descriptor, link radii, and active-link set match the current build. The split policy is also part of this descriptor; alternative policies are separate cache settings; descriptor matches reuse geometry bounds, not scene-validity labels. For a parent interval I , let I_d^- and I_d^+ be the two children obtained by splitting joint dimension d at its midpoint. Define the cumulative downstream envelope volume as

$$\mathcal{V}(I) := \sum_\ell \text{vol}(B_\ell(I)).$$

One implemented schedule chooses depth-wise split dimensions that minimize the worst-child cumulative envelope volume,

$$d^* = \arg \min_d \max\{\mathcal{V}(I_d^-), \mathcal{V}(I_d^+)\}.$$

Depth-synchronous schedules keep replay deterministic; if the scheduled dimension is unavailable at a node, the runtime falls back to the widest valid dimension. Candidate dimensions are filtered by active-link reachability: splitting a joint that cannot affect any retained endpoint cannot tighten the downstream envelopes and is skipped for that descriptor.

b) Sparse interval staging: Deep cells need not materialize every ancestor. A planning cell can carry its exact joint interval, split counts, and descriptor fingerprint; if an exact replay record is absent, the interval is materialized on demand under the same descriptor and then validated against the scene. Scene labels remain only in the forest or partition layer and are never inferred from a descriptor match.

B. Runtime Use and Validation Boundary

LECT serves local box materialization; graph search uses the scene forest. Given a joint interval, the runtime replays or materializes endpoint evidence, derives the requested link representation, and applies the current scene-validation policy. In the evaluated pipeline, AABB broadphase precedes SupportHull/GJK refinement. Parent or child records can accelerate materialization, but replayed evidence alone is not a collision-free certificate. When the required child records are present, their hulls can tighten an ancestor envelope; the reconstructed record remains representation-dependent and must still pass the current scene-validation policy. Directional records are staged filters: a joint interval first passes the padded AABB broadphase, and only the remaining close obstacle candidates reach slab or SupportHull/GJK refinement. If no exact replay record exists, the interval is materialized on demand and then handled by the same validation path, so a cache miss changes cost, not semantics or validation responsibility.

C. Storage and Warm-Build Semantics

Persistence is selective. Endpoint records and derived link-envelope records are the reusable evidence units. IFK+AABB records can be regenerated on demand, whereas slab or SupportHull records are stored only when their additional rejection power justifies lookup and storage cost. The prewarm pass fills target leaf envelope records and derives ancestor hulls from child evidence. This avoids recomputing direct forward kinematics independently at every ancestor depth. In a warm build, a cache hit reconstructs the requested link envelopes; the scene builder then checks those envelopes under the current scene-validation policy. Warm builds still choose and grow scene-specific boxes; persisted box labels are not imported as certificates. The registered Shelf replay is read-only during timing, so the replay store does not accumulate new evidence during those measurements. Random-scene cache growth is treated separately as part of the per-robot profile.

V. RapidBoxForest Construction

Fig. 2 illustrates the planning layer. RBF builds a scene-specific set of policy-accepted joint boxes and stores their adjacency as a typed corridor graph. The forest-graph pair is the scene cache: while the obstacle set is unchanged, later queries use box membership and graph search instead of rebuilding the cover. LECT remains a separate kinematics-keyed evidence store; every policy-accepted scene box is still checked against the current obstacles.

A box is an axis-aligned joint interval $B = [l^{(1)}, u^{(1)}] \times \dots \times [l^{(n)}, u^{(n)}] \subseteq \mathcal{Q}$. A *policy-accepted box* is policy-relative: it has passed the configured scene-validation policy. When the policy is conservative link-envelope disjointness, this acceptance is a collision-free certificate; under a non-conservative filter, the box is only a planning region, and any path segment relying on it must pass query validation. Cells that do not pass the policy are not inserted into the forest; selected blocked or inconclusive domains are retained for local refinement or later update.

A. Construction Objective

Given the joint-limit box $\mathcal{Q}$ and obstacle set $\mathcal{O}$, the build stage constructs

$$\mathcal{F} = \{B_1, \dots, B_N\}, \quad B_i \subseteq \mathcal{Q},$$

where each B_i satisfies the scene-validation policy. This budgeted set is concentrated around task anchors, query-relevant frontiers, and connectors. The reusable output is the pair $(\mathcal{F}, G)$, where G records adjacency and auxiliary transitions.

For boxes

$$B_i = \prod_{d=1}^n [l_i^{(d)}, u_i^{(d)}], \quad B_j = \prod_{d=1}^n [l_j^{(d)}, u_j^{(d)}],$$

RBF treats them as connected when every coordinate overlaps or is separated by at most a small tolerance ε_c :

$$\max\{l_i^{(d)}, l_j^{(d)}\} \leq \min\{u_i^{(d)}, u_j^{(d)}\} + \varepsilon_c, \quad d = 1, \dots, n.$$

Only exact overlap or contact directly yields a contained box corridor. Tolerance-gap and explicit segment edges are graph-search aids, not certificates, unless they are converted into conservative box-corridor or finite-cover OBB-corridor regions; otherwise they require query validation.

B. Build Pipeline

The build proceeds through four stages. First, local materialization descends LECT around a seed and returns the first interval on that descent accepted by the scene-validation policy. Second, a leaf sweep classifies shallow cells and records blocked domains for possible refinement or later edits. Third, frontier growth adds boxes near anchors and query-relevant boundaries. Finally, consolidation merges or removes redundant boxes only when the resulting boxes still satisfy the same validation rule. Algorithms 1 and 2 define the two validation routines reused throughout this pipeline; frontier growth and consolidation repeat the same materialize-validate-connect

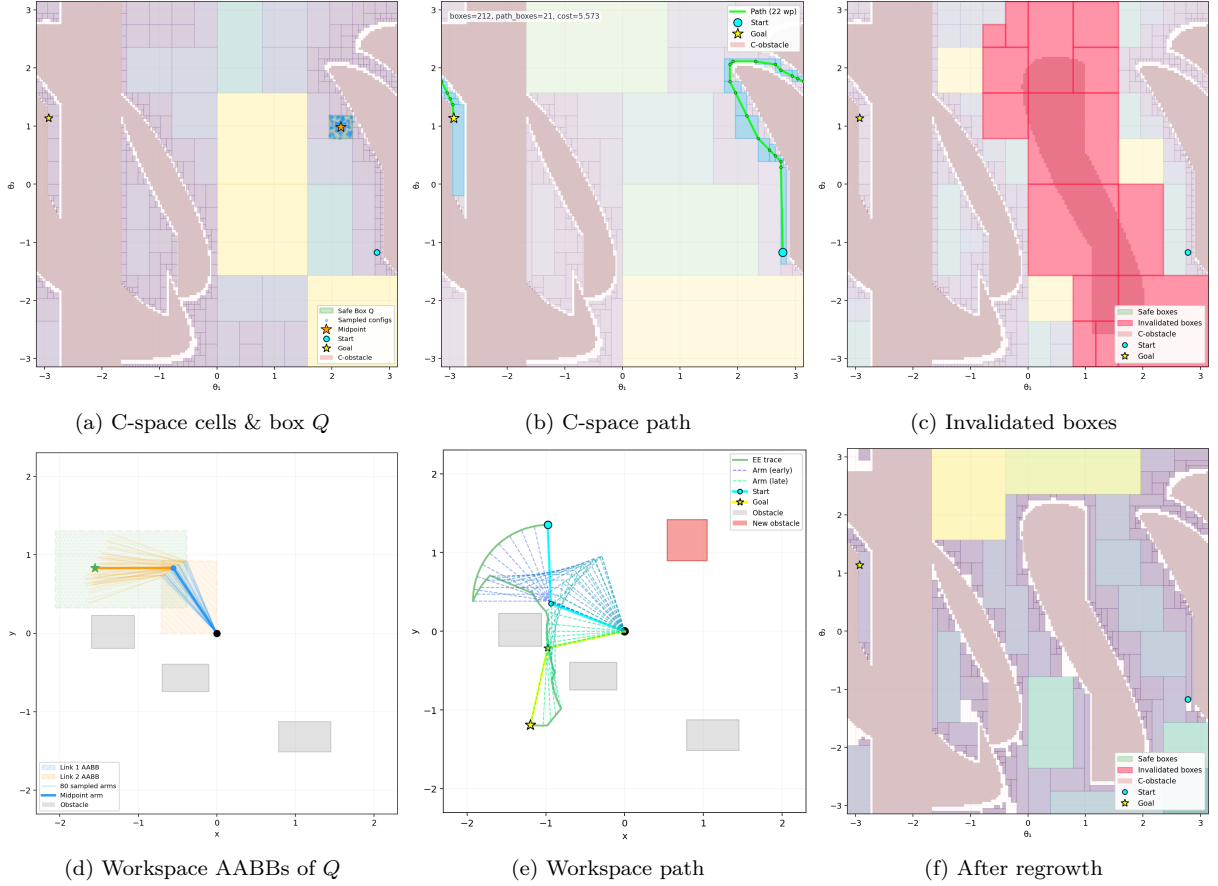


Fig. 2. Two-link schematic of the RBF build, query, and obstacle-update cycle. A highlighted C-space box maps to workspace AABBs; the box corridor supplies a candidate path; obstacle insertion invalidates intersecting boxes and triggers local refill without a full rebuild. The panels illustrate the build, query, and update logic rather than the robot geometry, workspace scale, or dimensionality of the experiments.

Algorithm 1 Local box materialization around a seed.

Input: Seed q , LECT T , obstacle set $\mathcal{O}$, depth cap $D_{\max}$
Output: Policy-accepted interval or failure

- 1: start from the root interval containing q
- 2: **while** depth $\leq D_{\max}$ **do**
- 3: replay or materialize current endpoint and link evidence
- 4: **if** the current interval passes the scene-validation policy **then**
- 5: **return** current interval
- 6: **end if**
- 7: split and descend to the child containing q
- 8: **end while**
- 9: **return** failure

Algorithm 2 Adaptive leaf sweep and local refinement.

Input: LECT T , obstacle set $\mathcal{O}$, sweep budget
Output: Initial forest $\mathcal{F}$ and retained collision domains $\mathcal{C}$

- 1: classify priority cells by replaying or materializing LECT evidence
- 2: insert cells that pass validation into $\mathcal{F}$; prune covered descendants
- 3: retain inconclusive or blocked cells in $\mathcal{C}$
- 4: refine only selected domains in $\mathcal{C}$ using local materialization
- 5: insert refined boxes that pass validation, remain inside their domain, and connect to the forest
- 6: **return** $\mathcal{F}, \mathcal{C}$

pattern under budget limits and do not introduce a separate validation primitive.

a) Implementation controls: Validation semantics and timings are governed by the build budget, LECT descriptor, endpoint and link representation, admitted edge types, and audit resolution. Materialization failures remain deferred domains for restricted refinement; frontier or repair witnesses are certificate-supported only after conversion into conservative box-corridor or finite-cover OBB-corridor regions. Lower-level scheduling constants are fixed in the artifact manifest.

C. Corridor Query and Certificate

After consolidation, G has policy-accepted vertices and edge types $E_{\cap}$ (overlap/contact), E_{ε} (tolerance gap), E_s (segment), E_o (finite-cover OBB corridor), and E_{π} (repair witness). Only $E_{\cap}$, E_o , and E_{π} can be certified corridor or portal edges, and only when backed by conservative region checks; E_{ε} , E_s , and other outputs require query validation. The edge label alone is therefore not a certificate.

Given start and goal configurations, the query locates containing boxes or configured anchor boxes and searches G . The certificate applies only to returned paths contained in conservative box regions or finite-cover OBB-corridor regions. Theorem 1 states the box-region case; Section III-C gives the finite-cover certificate for OBB-corridor edges. All other returns are query-validated re-

sults, not conservative-region proofs.

Theorem 1 (Conditional conservative box-corridor path). *Let $\mathcal{F}$ be a forest whose boxes have each passed conservative link-envelope validation against obstacle set $\mathcal{O}$. If a path $\gamma : [0, 1] \rightarrow \mathcal{Q}$ is continuous and satisfies*

$$\gamma([0, 1]) \subseteq \bigcup_{B \in \mathcal{F}} B,$$

then every configuration on γ is collision-free with respect to the collision model in Assumption 1.

Proof. For any $s \in [0, 1]$, the containment assumption gives a conservatively validated box $B \in \mathcal{F}$ with $\gamma(s) \in B$. Because B has passed conservative link-envelope validation, every conservative link envelope for B is disjoint from $\mathcal{O}$. Because each such envelope contains the link geometry for every configuration inside B (Corollary 1), the robot geometry at $\gamma(s)$ is also disjoint from $\mathcal{O}$. Since s was arbitrary, the whole path is collision-free. $\square$

Scope. The theorem establishes collision-freedom only for paths contained in conservatively validated regions. It does not establish coverage or completeness: finite depth, axis-aligned boxes, envelope overapproximation, and bridge or repair limits can miss paths. Paths using uncovered space, residual non-corridor segments, smoothing, or boxes admitted by non-conservative filters are accepted only after independent query validation.

D. Dynamic Obstacle Updates

For obstacle edits, robot kinematics and compatible LECT records stay fixed; scene labels, forest membership, and graph edges are updated locally. Deletions may promote cached blocked domains after edited-scene validation. Insertions invalidate conflicting boxes and scene-local typed witnesses, then run bounded local refill. The update is local by design: it updates the represented forest rather than exhaustively rediscovering every newly collision-free region.

Algorithm 3 Obstacle deletion promotion and insertion refill.

Input: Forest $\mathcal{F}$, typed graph G , collision-domain cache $\mathcal{C}$, edit (Δ^-, Δ^+)

Output: Updated forest, typed graph, and collision-domain cache

```

1: if  $\Delta^- \neq \emptyset$  then
2:   recheck cached blocked domains affected by deleted obstacles
3:   promote only domains that pass edited-scene validation and
   adjacency checks
4: end if
5: if  $\Delta^+ \neq \emptyset$  then
6:   remove boxes and scene-local typed witnesses hit by inserted
   obstacles
7:   run a bounded local leaf sweep inside each invalidated box
8:   insert replacement boxes that pass validation and connect to
   the surviving forest
9: end if
10: revalidate affected typed edges and update components
11: return updated  $(\mathcal{F}, G, \mathcal{C})$ 

```

After an update, retained or inserted boxes must satisfy the edited-scene validation policy, and revalidated typed edges must keep their original semantics. If the updated

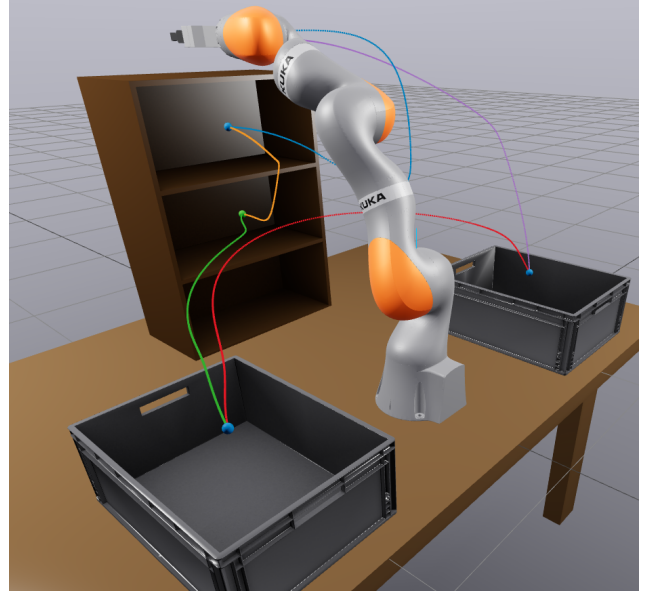


Fig. 3. Canonical Marcucci Shelf+IIWA scene with the fixed saved five-anchor query cycle.

graph cannot provide a contained conservative corridor, the candidate path is accepted only after independent query validation.

VI. Experiments

The experiments use three declared scopes: a registered Shelf+IIWA profile, saved random-scene catalogs, and maintenance-only scene-cache updates. Tables and trade-off plots are grouped for compact presentation, but the discussion follows that scope order. The Shelf+IIWA study uses the KUKA LBR iiwa shelf-workcell benchmark and the manifest-registered RBF configuration; *registered* means fixed in the artifact manifest before table extraction. Planning rows share success, audit, final-simplification, online-timing, and path-ratio conventions; offline curves sweep planner budget explicitly. Random scenes and queries are saved before planner execution, so methods share the same start-goal and obstacle instances within each scope.

TABLE II
Shelf+IIWA RBF query-validated profile and mechanism ablations.

Case	Build	Online/q	Amort. 5q	L/L^*
RBF baseline	0.166 [0.163, 0.170]	0.130 [0.104, 0.150]	0.164 [0.137, 0.183]	1.16 [1.08, 1.38]
Critical sample	0.282 [0.280, 0.285]	0.134 [0.103, 0.157]	0.190 [0.161, 0.213]	1.14 [1.08, 1.38]
Link AABB	0.141 [0.139, 0.143]	0.123 [0.099, 0.140]	0.151 [0.128, 0.168]	1.19 [1.08, 1.47]
Live HIFK-5	7.006 [6.947, 7.068]	0.234 [0.170, 0.278]	1.639 [1.576, 1.687]	1.14 [1.08, 1.39]
SupportHull only	0.349 [0.287, 0.363]	0.232 [0.155, 0.283]	0.296 [0.220, 0.352]	1.16 [1.08, 1.38]

Notes: Median $[Q_1, Q_3]$, s. Amort. 5q is Build/5 + Online/q; Online/q excludes final simplification and audit; L/L^* uses success-only query-level 0.01-rad audited references.

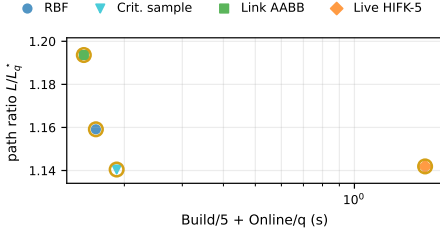


Fig. 4. Shelf+IIWA RBF mechanism trade-off. Query-validated mechanism rows are plotted at Build/5 + Online/q versus L/L_q^* ; gold rings mark the rows listed in Table II.

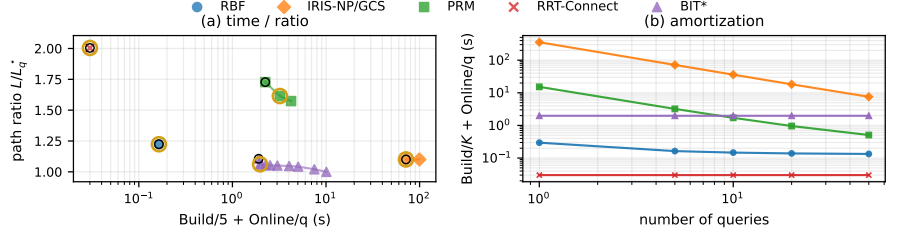


Fig. 5. Shelf+IIWA cross-algorithm trade-off. Black rings mark the first checkpoints with audited success on the saved five-query set. The horizontal coordinate is Build/5 + Online/q for methods with reusable builds and Online/q for single-query rows.

TABLE III
Shelf+IIWA audited-success per-query timing and path-quality comparison.

Query	RBF b100 Build 0.166 s		IRIS-NP/GCS Build 355.425 s		PRM Build 20.040 s		RRT-Connect No reusable build		BIT* No reusable build	
	T (s)	L/L^*	T (s)	L/L^*	T (s)	L/L^*	T (s)	L/L^*	T (s)	L/L^*
AS→TS	0.081 [0.078, 0.087]	1.39 [1.34, 1.48]	0.449 [0.449, 0.449]	1.44 [1.44, 1.44]	0.198 [0.147, 0.248]	1.61 [1.42, 1.73]	0.034 [0.024, 0.052]	2.27 [2.05, 2.72]	4.004 [4.002, 4.004]	1.06 [1.02, 1.08]
TS→CS	0.163 [0.158, 0.170]	1.12 [1.09, 1.14]	0.269 [0.269, 0.269]	1.26 [1.26, 1.26]	0.173 [0.118, 0.312]	2.99 [2.11, 3.22]	0.054 [0.021, 0.089]	3.98 [2.80, 5.09]	10.138 [10.004, 10.386]	1.09 [1.04, 1.11]
CS→LB	0.109 [0.104, 0.118]	1.47 [1.32, 1.82]	0.381 [0.381, 0.381]	1.07 [1.07, 1.07]	0.136 [0.120, 0.195]	1.49 [1.39, 2.09]	0.032 [0.027, 0.070]	2.20 [1.75, 2.51]	7.504 [7.504, 7.725]	1.03 [1.01, 1.06]
LB→RB	0.132 [0.128, 0.141]	1.15 [1.07, 1.25]	0.786 [0.786, 0.786]	1.07 [1.07, 1.07]	0.164 [0.091, 0.399]	1.18 [1.09, 1.27]	0.001 [0.001, 0.002]	1.29 [1.27, 1.32]	1.654 [1.654, 1.654]	1.02 [1.02, 1.03]
RB→AS	0.143 [0.138, 0.150]	1.04 [1.03, 1.05]	0.527 [0.527, 0.527]	1.01 [1.01, 1.01]	0.252 [0.096, 0.295]	1.23 [1.17, 1.24]	0.001 [0.001, 0.001]	1.07 [1.03, 1.10]	0.204 [0.204, 0.204]	1.02 [1.01, 1.03]

Notes: Median $[Q_1, Q_3]$, s. T = Online/q, excluding final simplification and audit; L/L^* uses the success-only query-level 0.01-rad audited reference path. AS, TS, CS, LB, and RB denote the approach-side anchor, two shelf-side anchors, and the left and right bin anchors.

a) Protocol: Timings are wall-clock measurements on an Ubuntu 22.04 workstation with an Intel i5-12600KF CPU and 31 GiB memory. The generated tables and figures are tied to provenance manifests that record sources, thread and simplification policies, checksums, and artifact-integrity checks. The manifests record an 8-thread budget for RBF and LECT work and matching math-library thread limits. OMPL RRT-Connect, PRM, and BIT* are single-threaded per-query invocations here; table Online/q entries are not scene-batch totals. Table headers include a reusable-build term only for methods that construct a persistent object before queries: RBF, PRM, and the Shelf IRIS-NP/GCS row (nonlinear-programming IRIS followed by GCS optimization). RRT-Connect and BIT* are reported as per-query timing baselines without a reusable-build term.

Success requires query validation with the fixed 0.01-rad segment step. Online/q denotes per-query online time and excludes final simplification and audit time. When the optional per-query audit column (Audit/q) is present,

the audit-complete reference is Online/q plus the fixed 0.010 s final-simplification budget plus Audit/q. Planning-table entries are medians with $[Q_1, Q_3]$ quartiles unless a caption or note states otherwise. Path-ratio columns divide audited returned length by the success-only, query-level 0.01-rad audited reference unless a caption or note states otherwise; lower is better. Mechanism-study tables use the aggregation stated in their notes. The main planning tables summarize query-validated outcomes; they are not conservative-corridor certificates, repair-free results, or parameter-sensitivity studies. The reported planning rows instantiate Assumption 1: active rounded links are checked against $\mathcal{O}$. Self-collision, attached objects, grippers, and task-specific forbidden regions require additional scene checks before using the certificate.

b) Cache and profile boundary: Warm-cache rows include compatible LECT lookup and evidence reads. Snapshot creation and loading costs, resident memory, and offline setup amortization are outside those rows. Here d23 denotes the registered depth-23 LECT snap-

shot: 16,777,215 records, 12.75 GiB on disk, and 958.079 s prewarm time. Except for Live HIFK-5, Table II rows replay compatible d23 evidence; the Live HIFK-5 row materializes HIFK-5 evidence live. The b100 label denotes the registered 100-box Shelf operating point. The baseline row uses the SupportHull representation with the standard AABB broadphase. Cache reuse requires matching robot model, endpoint source, link representation, split-depth descriptor, radii, and active-link set. Timing remains audit-separated. Bridge or repair witnesses become certificate-supported only after conversion into conservative box-corridor or finite-cover OBB-corridor edges; all remaining witnesses are reported only as query-validated transitions.

A. Shelf+IIWA Registered Profile Study

Fig. 3 shows the Shelf+IIWA scene and fixed five-anchor query cycle AS-TS-CS-LB-RB-AS; the anchor AS is the approach-side anchor. TS and CS are shelf-side target anchors, and LB and RB are the left and right bin anchors. Table II and Fig. 4 report the manifest-registered b100 profile over seeds 0–7. The five-anchor cycle is used for mechanism ablations and profile selection; the saved random-scene catalogs below test broader start-goal variation. The table reports build time, Online/q, five-query amortized time $\text{Build}/5 + \text{Online}/q$, and path ratio. The Critical sample row uses the same planner and fixed-step query validation; its endpoint approximation is treated as a non-certifying filter rather than as a conservative endpoint certificate.

The registered RBF baseline uses d23 SupportHull: Build 0.166 s, Online/q 0.130 s, and Amort. 5q 0.164 s; Table II gives quartiles. The Live HIFK-5 row has a 7.006 s live-evidence build; the other rows replay d23 evidence.

After corridor conversion, the Shelf manifest records no residual ordinary (non-corridor) segments for the 40 OBB-corridor paths in Table II. The representation rows compare cost: Link AABB lowers Online/q from 0.130 s to 0.123 s at a similar path ratio; SupportHull-only raises five-query time from 0.164 s to 0.296 s.

The Shelf profile results should therefore be read as query-validated pipeline results rather than pure box-corridor certificates. The manifest-registered query pipeline produced 40/40 audited paths. With query bridge, repair, shortcutting, and final simplification all disabled, a compound ablation produced 0/40; it identifies where the conservative-only pipeline stops without isolating single-stage effects.

B. Shelf Cross-Algorithm Context

The cross-algorithm shelf study uses the same five queries, 0.01-rad audit segment step, and 0.010 s final-simplification budget for RBF, IRIS-NP/GCS, PRM, RRT-Connect, and BIT*. RBF uses the b100 profile and per-query Online/q; PRM and IRIS-NP/GCS report reusable builds, while RRT-Connect and BIT* are single-query rows with no reusable-build term. BIT* checkpoints

are offline quality-time references, not online stopping rules.

Under these definitions, reusable builds are 0.166 s (RBF), 20.040 s (PRM), and 355.425 s (IRIS-NP/GCS). The RBF build excludes d23 prewarm, snapshot loading costs, and resident memory. In the per-query T columns, RRT-Connect is faster than RBF on all five Shelf queries but has higher, hence worse, L/L^* ; BIT* is slower than RBF but has lower L/L^* . This is a scoped timing-quality comparison for the saved five-query cycle, not a universal amortization claim.

C. Random Multi-Robot Scenes

The random-scene study uses saved random-scene catalogs: each scene has ten pre-generated queries shared by every method. Across KUKA LBR iiwa, UR5, and Panda, the medium and hard catalogs over ten seeds contain 600 saved query instances before the method-independent pruning rule below is applied. For each robot and seed, medium and hard rows share the saved start-goal set; hard rows append saved obstacle prefixes instead of regenerating unrelated scenes. Before any method summary is computed, a fixed manifest rule removes direct-line cases where the start-goal segment validates directly. This pruning affects reported summaries only; the stored catalogs still retain ten queries per scene. The trade-off plot uses $\text{Build}/5 + \text{Online}/q$ for reusable-build rows and Online/q otherwise, as a fixed five-query display point; K is a reuse horizon, not a post-pruning retained-query count. Table IV and Fig. 6 summarize the operating points and corresponding checkpoint curves. RBF builds one query-agnostic forest per scene and reuses it for ten queries; RBF random rows use LECT caches and anchors tied to each robot rather than Shelf-specific assets. Each RBF row in Table IV uses the OBB-corridor profile and passes the fixed audit on the reported saved query set. Repair witnesses contribute to accepted paths only after conversion into finite-cover C-space OBB-corridor regions; the generated summaries list no residual ordinary (non-corridor) segments for those RBF rows. PRM is built once per scene, whereas RRT-Connect and BIT* solve each query independently. The archived IRIS-NP/GCS saved-catalog run is a feasibility diagnostic: its strict saved-catalog success was too low to support comparable Online/q and path-ratio rows. The table-generation manifest therefore excludes it from Table IV and Fig. 6; the comparison reports Online/q and audited path ratios for RBF, PRM, RRT-Connect, and BIT* under shared catalogs. RRT-Connect has the lowest median Online/q in five of the six reported operating points; Panda-Hard is the exception. Its median audited path ratio is higher, hence worse, than RBF in all six. With scene-forest reuse, RBF has lower median Online/q than BIT* in all six cases; L/L_q^* matches BIT* to table precision in the two IIWA rows and is higher in the four UR5/Panda rows.

TABLE IV
Saved-catalog random-scene operating points.

Scenario	RBF			PRM			RRT-Connect		BIT*		
	Build	Online/q	L/L_q^*	Build	Online/q	L/L_q^*	Online/q	L/L_q^*	Online/q	L/L_q^*	
IIWA-Medium	0.075 [0.058, 0.083]	0.009 [0.007, 0.011]	1.00 [1.00, 1.03]	3.567 [3.007, 3.933]	0.038 [0.014, 0.070]	1.03 [1.02, 1.12]	0.002 [0.001, 0.002]	1.02 [1.01, 1.05]	0.107 [0.087, 0.154]	1.00 [1.00, 1.01]	
IIWA-Hard	0.097 [0.086, 0.105]	0.014 [0.011, 0.015]	1.00 [1.00, 1.03]	4.175 [3.408, 5.637]	0.048 [0.021, 0.102]	1.03 [1.02, 1.08]	0.002 [0.002, 0.003]	1.03 [1.00, 1.07]	0.129 [0.097, 0.186]	1.00 [1.00, 1.01]	
UR5-Medium	0.262 [0.254, 0.270]	0.075 [0.020, 0.086]	1.05 [1.00, 1.14]	3.124 [2.966, 3.418]	0.323 [0.085, 0.426]	1.02 [1.00, 1.12]	0.023 [0.002, 0.049]	1.23 [1.06, 1.54]	0.134 [0.101, 0.167]	1.01 [1.00, 1.08]	
UR5-Hard	0.276 [0.264, 0.284]	0.083 [0.043, 0.108]	1.05 [1.01, 1.14]	2.488 [2.325, 3.048]	0.419 [0.129, 0.437]	1.06 [1.01, 1.19]	0.048 [0.008, 0.090]	1.36 [1.14, 1.58]	0.154 [0.115, 0.171]	1.03 [1.00, 1.13]	
Panda-Medium	0.250 [0.218, 0.265]	0.028 [0.013, 0.039]	1.05 [1.02, 1.09]	3.431 [3.364, 4.180]	0.066 [0.038, 0.159]	1.06 [1.03, 1.08]	0.005 [0.002, 0.011]	1.22 [1.12, 1.35]	0.116 [0.092, 0.152]	1.00 [1.00, 1.00]	
Panda-Hard	0.224 [0.221, 0.237]	0.043 [0.021, 0.053]	1.05 [1.01, 1.16]	2.337 [1.627, 3.978]	0.054 [0.034, 0.126]	1.10 [1.05, 1.16]	0.056 [0.019, 0.136]	1.25 [1.14, 1.37]	0.164 [0.123, 0.214]	1.01 [1.00, 1.09]	

Notes: Median $[Q_1, Q_3]$, s. Build is a per-scene reusable-build term for RBF and PRM; RRT-Connect and BIT* are single-query rows. Online/q excludes final simplification and audit; L/L_q^* uses success-only query-level 0.01-rad audited references for the reported saved queries.

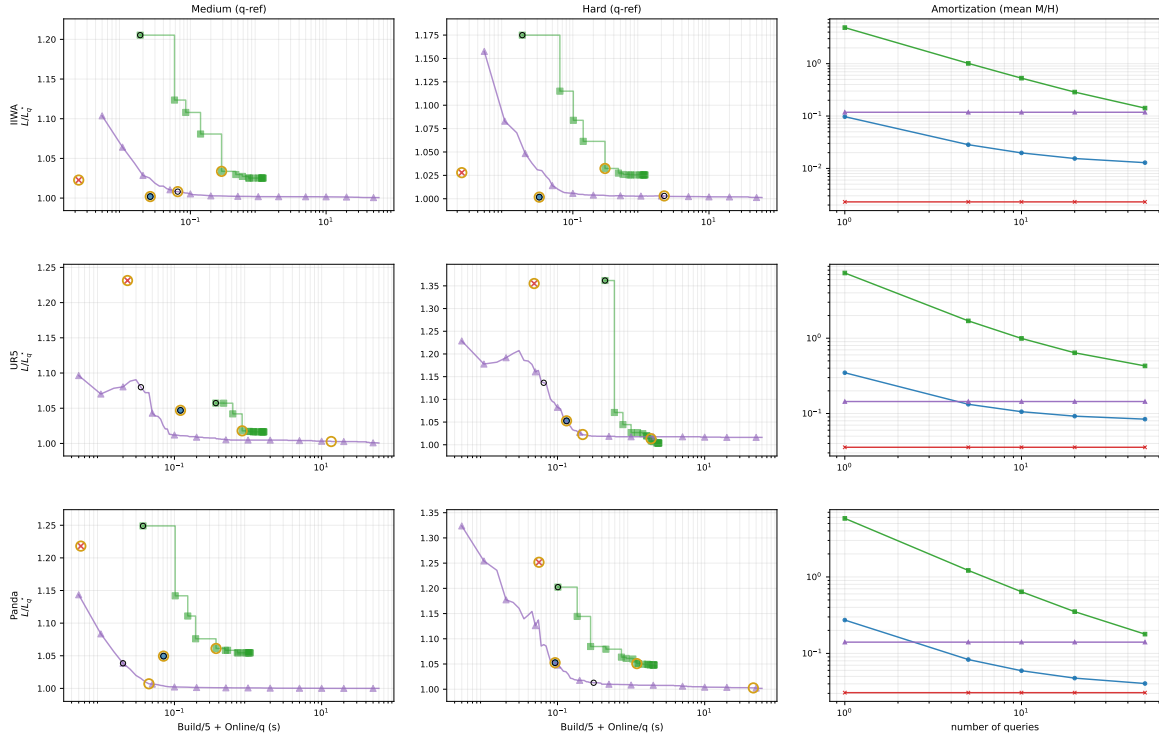


Fig. 6. Saved-catalog random-scene trade-off. Left and middle panels show a fixed five-query display point, Build/5 + Online/q, for medium and hard rows; the right panel varies the amortization count K in Build/ K + Online/q. Methods without a reusable-build term are plotted at Online/q. Rings mark the first checkpoints with audited success on the reported saved query set (black) and later $\geq 1\%$ path-ratio improvements (gold); PRM and BIT* curves use offline checkpoints.

D. Mechanism Studies

Tables V and VI compare endpoint sources and envelope narrow phases. Table V reports endpoint volume as a ratio to the Analytical row at the same width and the sampled-union signed gap in millimeters. Negative gaps are diagnostic undercoverage relative to the finite sampled union, not certificate margins. Only IFK-AA and HIFK rows are certificate-backed; Critical sample, Analytical, and MC rows are diagnostic or reference rows rather than certificate evidence. Table VI motivates the AABF broadphase: SupportHull/GJK stores a compact support record, but wide boxes increase collision-test time when every obstacle reaches the GJK narrow phase.

E. Dynamic Updates

Table VII reports scene-cache maintenance only: its warm-build rows are scene-cache rebuilds, and no query bridge, connector, simplification, audit, or query timing is included.

TABLE VII
Adaptive leaf-sweep maintenance-only timing.

Operation	Time (ms)	Speedup over 15-obstacle build
Warm build (10 obstacles)	922.75 [897.57, 955.67]	–
Insert to 15 obstacles	6.06 [5.69, 6.54]	153.3–207.5×
Remove from 15 obstacles	0.35 [0.31, 1.65]	943.6–3354.0×
Warm build (15 obstacles)	958.79 [941.67, 1115.18]	–

Notes: Median $[Q_1, Q_3]$, ms. Maintenance only, not end-to-end replanning; speedup compares each update row with the 15-obstacle warm build.

TABLE V
Endpoint-envelope source comparison over fixed joint-box widths.

Source	Volume ratio					Time (μ s)					Gap (mm)				
	0.02	0.10	0.20	0.30	0.50	0.02	0.10	0.20	0.30	0.50	0.02	0.10	0.20	0.30	0.50
IFK-AA	1.09	1.52	2.30	3.53	8.61	5.48	6.45	6.58	6.81	7.35	—	—	—	—	—
HIFK-3	1.04	1.22	1.51	1.89	3.12	25.94	29.16	27.21	29.27	29.38	—	—	—	—	—
HIFK-5	1.02	1.13	1.30	1.51	2.07	70.40	73.82	73.15	75.92	72.32	—	—	—	—	—
Critical sample	1.00	1.00	1.00	0.99	0.98	8.68	10.03	11.56	12.69	16.33	-0.05	-1.17	-4.53	-10.77	-30.97
Analytical	1.00	1.00	1.00	1.00	1.00	6429.96	6602.25	6800.10	6927.12	7096.01	0.00	0.00	0.00	0.00	-4.49
MC	0.64	0.74	0.78	0.79	0.82	2251.74	11233.26	22352.95	33813.58	56835.57	-6.05	-32.68	-65.57	-88.87	-123.53

Notes: Volumes are normalized to Analytical at each width. IFK-AA and HIFK are the certificate-backed rows; Critical sample, Analytical, and Monte Carlo (MC) are diagnostic or reference rows. Gap is the sampled-union signed gap (mm); dashes mark rows where the diagnostic is omitted.

TABLE VI
Link-envelope representation comparison over fixed joint-box widths.

Envelope	Volume						Build (μ s)						Test (μ s)					
	0.02	0.05	0.10	0.20	0.30	0.50	0.02	0.05	0.10	0.20	0.30	0.50	0.02	0.05	0.10	0.20	0.30	0.50
Link AAB	0.0136	0.0201	0.0384	0.147	0.508	4.61	0.138	0.152	0.167	0.166	0.171	0.178	0.586	0.605	0.601	0.604	0.630	0.790
SupportHull	0.000359	0.00266	0.0143	0.106	0.449	4.54	0.205	0.214	0.218	0.219	0.217	0.250	12.520	12.850	12.267	12.223	12.659	13.026

Notes: Means over fixed-width boxes. Test time pools far-separated probes and local obstacle-overlap probes; build time excludes endpoint time.

A diagnostic audit on locally updated forests and matching freshly rebuilt forests, each without the query bridge and repair stages, produced 0/8 paths in both diagnostic configurations. The 0/8 result is a scope check: without bridge and repair, the maintenance-only configuration is not an end-to-end planning pipeline, so the row should not be read as post-edit query success or a replanning benchmark.

VII. Conclusion

RBF is a link-envelope-guided box-forest planner for multi-query manipulation. It uses axis-aligned boxes as the reusable region type: endpoint-to-link swept-capsule bounds provide the validation test, policy-accepted boxes form a scene-level forest and typed corridor graph, and LECT separates reusable kinematics-keyed evidence from scene labels. Certificates apply only to conservative box unions or finite-cover OBB-corridor unions; paths that are repaired, shortened, post-processed, or derived from filter-only evidence require independent query validation.

Under saved catalogs, audit-separated timing, and explicit cache accounting, warm-cache depth-23 LECT replay on Shelf+IIWA yields a 0.166 s median reusable build and 0.130 s median Online/q, whereas live HIFK-5 materialization takes 7.006 s. In saved random scenes, RRT-Connect is faster in five cases but has higher path ratios than RBF; against BIT*, RBF has lower median Online/q in all six, while BIT* has lower path ratios in the four UR5/Panda cases. The dynamic-update rows measure scene-cache maintenance rather than full update-and-query performance. Within these scopes, the results characterize RBF as a reusable-region construction method for repeated queries, not as a complete replacement for single-query planners or full-scene replanning. The remaining limitations follow from the same boundaries. Certificates do not cover repaired, shortened, post-processed, or filter-only paths unless those paths are converted into conservative regions. LECT snapshot creation and loading costs, resident memory, and memory break-even analyses are not reported. Validation beyond active-link obstacle

collision, including self-collision, payload and gripper models, and task-specific regions, remains future work.

References

- [1] R. Deits and R. Tedrake, "Computing large convex regions of obstacle-free space through semidefinite programming," in *Algorithmic Foundations of Robotics XI*, 2015, pp. 109–124.
- [2] T. Marcucci, M. Petersen, D. von Wrangel, and R. Tedrake, "Motion planning around obstacles with convex optimization," *Sci. Robot.*, vol. 8, no. 84, 2023.
- [3] T. Marcucci, J. Umenberger, P. Parrilo, and R. Tedrake, "Shortest paths in graphs of convex sets," *SIAM J. Optim.*, vol. 34, no. 1, pp. 507–532, 2024.
- [4] S. M. LaValle, *Planning Algorithms*. Cambridge University Press, 2006.
- [5] —, "Rapidly-exploring random trees: A new tool for path planning," Iowa State University, Tech. Rep. TR 98-11, 1998.
- [6] L. E. Kavraki, P. Svestka, J.-C. Latombe, and M. H. Overmars, "Probabilistic roadmaps for path planning in high-dimensional configuration spaces," *IEEE Trans. Robot. Autom.*, vol. 12, no. 4, pp. 566–580, 1996.
- [7] S. Karaman and E. Frazzoli, "Sampling-based algorithms for optimal motion planning," *Int. J. Robot. Res.*, vol. 30, no. 7, pp. 846–894, 2011.
- [8] A. Amice, H. Dai, P. Werner, A. Zhang, and R. Tedrake, "Finding and optimizing certified, collision-free regions in configuration space for robot manipulators," in *Algorithmic Foundations of Robotics XV*. Springer, 2022, pp. 328–348.
- [9] M. Petersen and R. Tedrake, "Growing convex collision-free regions in configuration space using nonlinear programming," 2023. [Online]. Available: <https://arxiv.org/abs/2303.14737>
- [10] P. Werner, A. Amice, T. Marcucci, D. Rus, and R. Tedrake, "Approximating robot configuration spaces with few convex sets using clique covers of visibility graphs," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 10 359–10 365.
- [11] D. Blackmore and M. C. Leu, "Analysis of swept volume via Lie groups and differential equations," *Int. J. Robot. Res.*, vol. 11, no. 6, pp. 516–537, 1992.
- [12] K. Abdel-Malek, J. Yang, D. Blackmore, and K. Joy, "Swept volumes: foundation, perspectives, and applications," *Int. J. Shape Model.*, vol. 12, no. 1, pp. 87–127, 2006.
- [13] S. Gottschalk, M. C. Lin, and D. Manocha, "OBBTree: A hierarchical structure for rapid interference detection," in *ACM SIGGRAPH*, 1996.
- [14] E. Larsen, S. Gottschalk, M. C. Lin, and D. Manocha, "Fast proximity queries with swept sphere volumes," University of North Carolina at Chapel Hill, Tech. Rep. TR99-018, 1999.
- [15] E. G. Gilbert, D. W. Johnson, and S. S. Keerthi, "A fast procedure for computing the distance between complex objects in three-dimensional space," *IEEE J. Robot. Autom.*, vol. 4, no. 2, pp. 193–203, 1988.
- [16] G. v. d. Bergen, "A fast and robust GJK implementation for collision detection of convex objects," *J. Graph. Tools*, vol. 4, no. 2, pp. 7–25, 1999.
- [17] J. Pan, S. Chitta, and D. Manocha, "FCL: A general purpose library for collision and proximity queries," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2012, pp. 3859–3866.
- [18] R. E. Moore, R. B. Kearfott, and M. J. Cloud, *Introduction to Interval Analysis*. SIAM, 2009.
- [19] L. Jaulin, M. Kieffer, O. Didrit, and E. Walter, *Applied Interval Analysis*. Springer, 2001.
- [20] J. M. Snyder, "Interval analysis for computer graphics," in *Proc. ACM SIGGRAPH*, 1992, pp. 121–130.
- [21] J.-P. Merlet, "Solving the forward kinematics of a Gough-type parallel manipulator with interval analysis," *Int. J. Robot. Res.*, vol. 23, no. 3, pp. 221–235, 2004.
- [22] —, "Interval analysis for certified numerical solution of problems in robotics," *Int. J. Appl. Math. Comput. Sci.*, vol. 19, no. 3, pp. 399–412, 2009.

- [23] S. Redon, Y. J. Kim, M. C. Lin, and D. Manocha, "Interactive and continuous collision detection for avatars in virtual environments," in *IEEE Virtual Reality Conference*, 2004, pp. 117–124.
- [24] S. Redon, M. C. Lin, D. Manocha, and Y. J. Kim, "Fast continuous collision detection for articulated models," *J. Comput. Inf. Sci. Eng.*, vol. 5, no. 2, pp. 126–137, 2005.
- [25] X. Zhang, S. Redon, M. Lee, and Y. J. Kim, "Continuous collision detection for articulated models using Taylor models and temporal culling," *ACM Trans. Graph.*, vol. 26, no. 3, p. 15, 2007.
- [26] F. Schwarzer, M. Saha, and J.-C. Latombe, "Adaptive dynamic collision checking for single and multiple articulated robots in complex environments," *IEEE Trans. Robot.*, vol. 21, no. 3, pp. 338–353, 2005.
- [27] J. A. Corrales, F. A. Candelas, and F. Torres, "Safe human–robot interaction based on dynamic sphere-swept line bounding volumes," *Robot. Comput. Integr. Manuf.*, vol. 27, no. 1, pp. 177–185, 2011.
- [28] R. Bohlin and L. E. Kavraki, "Path planning using lazy PRM," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2000, pp. 521–528.
- [29] C. M. Dellin and S. S. Srinivasa, "A unifying formalism for shortest path problems with expensive edge evaluations via lazy best-first search over paths with edge selectors," in *International Conference on Automated Planning and Scheduling (ICAPS)*, 2016.
- [30] E. Schmerling, L. Janson, and M. Pavone, "Optimal sampling-based motion planning under differential constraints: The driftless case," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2015.
- [31] D. Coleman, I. Sucan, S. Chitta, and N. Correll, "Reducing the barrier to entry of complex robotic software: A MoveIt! case study," *J. Softw. Eng. Robot.*, vol. 5, no. 1, pp. 3–16, 2014.
- [32] T. Lozano-Pérez, "Spatial planning: A configuration space approach," *IEEE Trans. Comput.*, vol. C-32, no. 2, pp. 108–120, 1983.
- [33] R. A. Brooks and T. Lozano-Pérez, "A subdivision algorithm in configuration space for findpath with rotation," *IEEE Trans. Syst. Man Cybern.*, vol. SMC-15, no. 2, pp. 224–233, 1985.
- [34] J. J. Kuffner and S. M. LaValle, "RRT-Connect: An efficient approach to single-query path planning," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2000, pp. 995–1001.
- [35] J. D. Gammell, S. S. Srinivasa, and T. D. Barfoot, "Batch Informed Trees (BIT*): Sampling-based optimal planning via the heuristically guided search of implicit random geometric graphs," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2015, pp. 3067–3074.
- [36] J. D. Gammell, T. D. Barfoot, and S. S. Srinivasa, "Informed sampling for asymptotically optimal path planning," *IEEE Trans. Robot.*, vol. 34, no. 4, pp. 966–984, 2018.
- [37] L. Janson, E. Schmerling, A. Clark, and M. Pavone, "Fast Marching Tree: A fast marching sampling-based method for optimal motion planning in many dimensions," *Int. J. Robot. Res.*, vol. 34, no. 7, pp. 883–921, 2015.
- [38] I. A. Şucan, M. Moll, and L. E. Kavraki, "The Open Motion Planning Library," *IEEE Robot. Autom. Mag.*, vol. 19, no. 4, pp. 72–82, 2012.
- [39] M. Moll, I. A. Şucan, and L. E. Kavraki, "Benchmarking motion planning algorithms: An extensible infrastructure for analysis and visualization," *IEEE Robot. Autom. Mag.*, vol. 22, no. 3, pp. 96–102, 2015.
- [40] N. Ratliff, M. Zucker, J. A. Bagnell, and S. Srinivasa, "CHOMP: Gradient optimization techniques for efficient motion planning," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2009, pp. 489–494.
- [41] M. Kalakrishnan, S. Chitta, E. Theodorou, P. Pastor, and S. Schaal, "STOMP: Stochastic trajectory optimization for motion planning," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2011, pp. 4569–4574.
- [42] J. Schulman, Y. Duan, J. Ho, A. Lee, I. Awwal, H. Bradlow, J. Pan, S. Patil, K. Goldberg, and P. Abbeel, "Motion planning with sequential convex optimization and convex collision checking," *Int. J. Robot. Res.*, vol. 33, no. 9, pp. 1251–1270, 2014.
- [43] M. Mukadam, J. Dong, X. Yan, F. Dellaert, and B. Boots, "Continuous-time Gaussian process motion planning via probabilistic inference," *Int. J. Robot. Res.*, vol. 37, no. 11, pp. 1319–1340, 2018.